NLU Assignment 2

Problem 1: Corpus Creation, Word2Vec & Semantic Analysis
Problem 2: Character-Level Name Generation using RNN Variants

CSL 7640: Natural Language Understanding

Abstract

This report covers two problems in natural language understanding. **Problem 1** describes the construction of a domain-specific text corpus from IIT Jodhpur PDF documents, the from-scratch implementation of Word2Vec in both CBOW and Skip-gram with Negative Sampling architectures using pure NumPy, and subsequent semantic analysis and embedding visualisation. The pipeline covers PDF extraction via `PyPDFLoader`, custom tokenisation without external libraries, systematic hyperparameter experimentation, nearest-neighbour retrieval, word analogy tasks, and 2D embedding projections via PCA and t-SNE. **Problem 2** presents a comparative study of three recurrent neural network architectures for character-level Indian name generation: a Vanilla RNN, a Bidirectional LSTM, and an RNN encoder–decoder with Bahdanau attention. All models are trained on 1000 synthetic Indian names and evaluated on novelty, diversity, qualitative realism, and learned character embedding structure.

Problem 1: Corpus Creation, Word2Vec & Semantic Analysis

1 Introduction

Distributed word representations—where each word is mapped to a dense, low-dimensional vector—have become a foundational component of modern NLP. Word2Vec, proposed by Mikolov et al., learns these representations by training a shallow neural network to predict either a target word from its context (CBOW) or context words from a target (Skip-gram). The resulting vectors capture semantic and syntactic regularities: words that appear in similar contexts are mapped to nearby points in the embedding space.

In this problem we build a complete Word2Vec pipeline from scratch. We first collect and preprocess textual data from three IIT Jodhpur PDF documents (academic regulations, course catalog, and curriculum), producing a cleaned, tokenised corpus. We then implement both CBOW and Skip-gram architectures with negative sampling using only NumPy—no deep learning or NLP libraries are used for the core model. Finally, we evaluate the learned embeddings through nearest-neighbour queries, word analogy experiments, and 2D visualisations.

2 Data Collection and Preprocessing

2.1 PDF Text Extraction

Three PDF documents from IIT Jodhpur were used as the data source, loaded using `PyPDFLoader` from the `langchain_community` library. Each PDF is read page-wise and all pages are concatenated into a single string per document.

Table 1: Raw document statistics after PDF extraction.

Document	Characters	Tokens (after cleaning)
doc1.pdf (Academic Regulations)	121,921	8,289
doc2.pdf (Course Catalog)	40,019	3,730
doc3.pdf (Curriculum)	246,994	22,174
Total	408,934	34,193

2.2 Preprocessing Pipeline

The preprocessing function `clean_and_tokenize` applies the following steps in sequence:

1. **Boilerplate removal:** URLs, email addresses, page numbers, page-of-page references, and form-feed characters are stripped using regular expressions.
2. **Non-ASCII removal:** All characters outside the ASCII range are replaced with spaces, eliminating Hindi text and other non-English scripts.
3. **Lowercasing:** The entire text is converted to lowercase.
4. **Character filtering:** All characters except lowercase letters, whitespace, and periods are replaced with spaces; multiple spaces are collapsed.
5. **Custom tokenisation:** A hand-written tokeniser (`my_tokenize_func`) splits text on whitespace using `str.split()`, strips leading/trailing punctuation via `string.punctuation`, retains only alphabetic tokens via `str.isalpha()`, converts to lowercase, removes stopwords (NLTK English stopword list), and discards single-character tokens. No external tokenisation library (e.g., `word_tokenize`) is used.

2.3 Corpus Statistics

After preprocessing, each document's cleaned text is saved individually (`corpus_1.txt`, `corpus_2.txt`, `corpus_3.txt`) and then merged into a single `final_corpus.txt` (356,318 bytes) with document-level separation.

Table 2: Corpus statistics after preprocessing.

Statistic	Value
Total documents	3
Total tokens	34,193
Vocabulary size	6,464

The top-5 most frequent words (excluding stopwords and words ≤ 2 characters) are: *lectures* (772), *course* (438), *learning* (382), *data* (351), and *computer* (255), reflecting the academic and technical nature of the source documents.

2.4 Sentence Preparation for Word2Vec

The merged cleaned text is split into sentences using NLTK's `sent_tokenize`, and each sentence is tokenised using the custom tokeniser. Sentences with fewer than 3 tokens are discarded. This produces **2,421 sentences** containing **33,586 tokens** for training.

3 Word2Vec Implementation

The Word2Vec model is implemented from scratch as a Python class (`Word2VecScrth`) using only NumPy. It supports both CBOW and Skip-gram architectures with negative sampling.

3.1 CBOW Architecture

In CBOW, the model predicts a target word w_t given its surrounding context words $\{w_{t-c}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_t$ within a window of size c. The context representation is the mean of the input embeddings of all context words:

$$\mathbf{h} = \frac{1}{|C|} \sum_{j \in C} \mathbf{W}_{\text{in}}[j] \tag{1}$$

where C is the set of context word indices and $\mathbf{W}_{\text{in}} \in \mathbb{R}^{V \times d}$ is the input embedding matrix.

3.2 Skip-gram Architecture

In Skip-gram, the model predicts each context word w_j given the centre word w_t. The hidden representation is simply the input embedding of the centre word:

$$\mathbf{h} = \mathbf{W}_{\text{in}}[t] \tag{2}$$

3.3 Negative Sampling Objective

Instead of a full softmax over the vocabulary, both architectures use negative sampling. For a positive sample w_+ and K randomly drawn negative samples $\{w_{n_1}, \ldots, w_{n_K}\}$, the objective maximises:

$$\mathcal{L} = \log \sigma(\mathbf{w}_+^\top \mathbf{h}) + \sum_{k=1}^{K} \log \sigma(-\mathbf{w}_{n_k}^\top \mathbf{h}) \tag{3}$$

where $\sigma(x) = 1/(1 + e^{-x})$ is the sigmoid function and $\mathbf{w}_i = \mathbf{W}_{\text{out}}[i]$ are the output embedding vectors. Negative samples are drawn from a noise distribution proportional to unigram frequency raised to the $\frac{3}{4}$ power:

$$P_{\text{noise}}(w_i) = \frac{f(w_i)^{3/4}}{\sum_{j=1}^{V} f(w_j)^{3/4}} \tag{4}$$

3.4 Gradient Updates

For each target/negative pair with label $\ell_i \in \{0, 1\}$:

$$s_i = \sigma(\mathbf{w}_i^\top \mathbf{h}) \tag{5}$$

$$g_i = \eta \cdot (\ell_i - s_i) \tag{6}$$

$$\mathbf{W}_{\text{out}}[i] \mathrel{+}= g_i \cdot \mathbf{h} \tag{7}$$

$$\Delta \mathbf{h} = \textstyle\sum_i g_i \cdot \mathbf{W}_{\text{out}}[i] \tag{8}$$

For CBOW, $\Delta \mathbf{h}$ is distributed equally to all context word input embeddings. For Skip-gram, it updates the single centre word embedding directly.

3.5 Training Details

- **Weight initialisation:** $\mathbf{W}_{\text{in}}$ initialised uniformly in $[-0.5/d,\ 0.5/d]$; $\mathbf{W}_{\text{out}}$ initialised to zero.
- **Learning rate decay:** Linear decay from η_0 to $\eta_0 \times 10^{-4}$ over all epochs.
- **Dynamic window:** Actual window size sampled uniformly from $[1, c]$ at each position.
- **Gradient clipping:** Dot products clipped to $[-6, 6]$ before the sigmoid.
- **OOV handling:** When a query word is not in the vocabulary, the closest vocabulary word by Levenshtein edit distance is used instead, with early stopping for 1-edit matches.

4 Hyperparameter Experiments

Three sets of experiments were conducted, each varying one hyperparameter while keeping the others fixed. All experiments used `min_count=3` and `epochs=3`.

4.1 Experiment 1: Embedding Dimension

Table 3: Effect of embedding dimension (window=5, neg=5, vocab=1,964).

Model	**Dim**	**Time (s)**	**Top-3 NN ("student")**
CBOW	50	9.9	lectures, data, algorithms
Skip-gram	50	29.4	academic, minimum, semester
CBOW	100	11.1	lectures, data, learning
Skip-gram	100	30.5	academic, semester, minimum
CBOW	200	11.6	lectures, data, design
Skip-gram	200	29.9	semester, credits, academic

Higher dimensions slightly increase CBOW training time but have minimal impact on Skip-gram. CBOW neighbours are dominated by high-frequency co-occurring words, while Skip-gram captures more structurally related terms.

4.2 Experiment 2: Context Window

Table 4: Effect of context window size (dim=100, neg=5).

Model	**Window**	**Time (s)**	**Top-3 NN ("student")**
CBOW	2	10.8	lectures, data, algorithms
Skip-gram	2	16.1	categories, credits, academic
CBOW	5	10.7	lectures, data, learning
Skip-gram	5	47.9	academic, semester, minimum
CBOW	8	10.1	lectures, learning, data
Skip-gram	8	38.4	dean, drc, shall

Larger windows capture broader, more topical relationships. At window=8, Skip-gram's neighbours for "student" shift towards institutional terms (*dean*, *drc*), reflecting wider administrative context.

4.3 Experiment 3: Negative Samples

Table 5: Effect of negative sample count (dim=100, window=5).

Model	**Neg**	**Time (s)**	**Top-3 NN ("student")**
CBOW	3	10.8	course, lectures, degree
Skip-gram	3	29.0	academic, semester, credits
CBOW	5	10.3	lectures, data, learning
Skip-gram	5	28.3	academic, semester, minimum
CBOW	10	10.2	research, algorithms, systems
Skip-gram	10	31.5	requirements, semester, regular

More negative samples provide stronger contrastive learning. At neg=10, CBOW's neighbours become more semantically diverse.

5 Final Model Training and Semantic Analysis

Final models were trained with `dim=100`, `window=5`, `neg=5`, `min_count=3`, and `epochs=5`.

Table 6: Final model configuration and training summary.

Parameter	**CBOW**	**Skip-gram**
Vocabulary size	1,964	1,964
Embedding dimension	100	100
Window size	5	5
Negative samples	5	5
Training epochs	5	5
Training time (s)	19.6	48.4

5.1 Nearest Neighbours

We query the top-3 nearest neighbours by cosine similarity for four words.

Table 7: Top-3 nearest neighbours for query words.

Query	**Rank**	**CBOW**	**Skip-gram**
research	1	student (0.9994)	papers
	2	lectures (0.9994)	announced
	3	networks (0.9993)	materialrelevant
student	1	systems (0.9995)	academics
	2	lectures (0.9995)	dean
	3	algorithms (0.9995)	thesis
phd	1	software (0.9953)	mtech
	2	type (0.9952)	decided
	3	introduction (0.9950)	tech
exam[†]	OOV — resolved to "team" (edit dist=2)		

[†]The word "exam" does not appear in the vocabulary (filtered by `min_count=3`); the OOV handler resolves it to "team" via Levenshtein distance.

The CBOW model produces neighbours with very high cosine similarities (> 0.999), indicating tightly clustered embeddings—a known behaviour on small corpora where context averaging smooths the embedding space. The Skip-gram model produces more semantically meaningful neighbours: "phd" maps to "mtech", "student" maps to "thesis" and "dean".

5.2 Word Analogies

We test four analogy queries of the form **A : B :: C : ?**, computed as $\mathbf{v}(B) + \mathbf{v}(C) - \mathbf{v}(A)$.

Table 8: Word analogy results (top prediction shown).

Analogy	**CBOW**	**Skip-gram**
UG : BTech† :: PG : ?	offered	mtech
bachelor : BTech† :: master : ?	offered	mtech
learning : classroom :: research : ?	lynch	laboratory
faculty : teaching :: student : ?	requirements	requirements

† "btech" is OOV and resolved to "tech" (edit dist=1).

The Skip-gram model correctly identifies "mtech" as the postgraduate counterpart of "tech" in both degree analogies. The CBOW model returns less meaningful results due to its smoother embedding space on this small corpus.

6 Visualisation

6.1 PCA and t-SNE Projections

The top 80 most frequent words' embeddings are projected to 2D using PCA and t-SNE (perplexity=30, 1000 iterations). In both projections, semantically related words (e.g., *course*, *lectures*, *learning*) tend to cluster together. The Skip-gram projections show more distinct local clusters than CBOW.

6.2 Semantic Category Clustering

Words are grouped into four categories—*People* (student, faculty, professor, ...), *Academics* (course, lecture, lab, ...), *Degrees* (btech, mtech, phd, ...), and *Metrics* (exam, grade, cgpa, ...)—and projected using both PCA and t-SNE. The Skip-gram model shows clearer inter-category separation than CBOW, consistent with its sharper, more discriminative embedding space.

7 Problem 1: Conclusion

On this small, domain-specific corpus (34K tokens, 2K vocabulary), CBOW trains $\sim$2.5$\times$ faster (19.6s vs. 48.4s) but produces overly uniform embeddings with cosine similarities exceeding 0.999. Skip-gram produces sharper, more semantically meaningful embeddings: "phd" maps to "mtech", "student" maps to "thesis", and the degree analogies resolve correctly. The edit-distance-based OOV resolution provides a practical fallback ("btech" $\rightarrow$ "tech") but can produce poor matches ("exam" $\rightarrow$ "team"), highlighting a limitation of purely string-based OOV handling.

Problem 2: Character-Level Name Generation using RNN Variants

1 Introduction

Character-level language models learn to predict the next character in a sequence, enabling them to generate plausible strings after training on a corpus of examples. In this problem we apply this idea to Indian name generation: given a dataset of names, the model should learn the phonotactic regularities—common prefixes such as *Aar-*, *Dev-*, *Shiv-*, suffixes such as *-esh*, *-raj*, *-vansh*, and consonant–vowel alternation patterns—and produce novel, realistic names.

We implement three architectures of increasing representational power: (i) a multi-layer Vanilla RNN, (ii) a Bidirectional LSTM, and (iii) an RNN encoder–decoder with Bahdanau-style attention. All three share the same hyperparameters and training procedure so that differences in output quality can be attributed to architectural choices.

2 Dataset Description

The training corpus (`TrainingNames.txt`) contains **1000 Indian names** synthetically constructed by combining common prefixes (e.g., *Aar*, *Adi*, *Dev*, *Krish*, *Shiv*, *Yash*) with common suffixes (e.g., *ansh*, *raj*, *veer*, *esh*, *yash*). All names are lowercased, yielding an alphabet of **21 unique characters**: `a, b, c, d, e, g, h, i, j, k, l, m, n, o, p, r, s, t, u, v, y`. Together with three special tokens (`<PAD>`, `<SOS>`, `<EOS>`), the vocabulary size is $V = 24$.

Name lengths range from 4 to 10 characters with a mean of 6.3. Each name is encoded as a pair of integer sequences: the input is prepended with `<SOS>` and the target is appended with `<EOS>`. Variable-length sequences within a mini-batch are right-padded with `<PAD>`.

3 Methodology

All three models share a learnable character embedding layer of dimension $d_e = 64$ that maps each of the $V = 24$ token indices to a dense vector, with the `<PAD>` index zeroed out.

3.1 Vanilla RNN

The Vanilla RNN uses a 2-layer Elman recurrent network. At each time step t:

$$\mathbf{h}_t = \tanh\big(\mathbf{W}_{xh}\,\mathbf{x}_t + \mathbf{W}_{hh}\,\mathbf{h}_{t-1} + \mathbf{b}_h\big) \tag{9}$$

$$\mathbf{y}_t = \mathrm{softmax}\big(\mathbf{W}_{hy}\,\mathbf{h}_t + \mathbf{b}_y\big) \tag{10}$$

where $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times d_e}$, $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$, $\mathbf{W}_{hy} \in \mathbb{R}^{V \times d_h}$. Dropout ($p = 0.2$) is applied after the embedding and after the RNN output.

Trainable parameters. Embedding (1,536) + RNN Layer 1 (24,832) + RNN Layer 2 (33,024) + Output (3,096) = **62,488**.

3.2 Bidirectional LSTM (BLSTM)

The BLSTM uses LSTM cells processing the sequence in both directions. For each direction, the LSTM gates at time step t are:

$$\mathbf{f}_t = \sigma\big(\mathbf{W}_f\,\mathbf{x}_t + \mathbf{U}_f\,\mathbf{h}_{t-1} + \mathbf{b}_f\big) \tag{11}$$
$$\mathbf{i}_t = \sigma\big(\mathbf{W}_i\,\mathbf{x}_t + \mathbf{U}_i\,\mathbf{h}_{t-1} + \mathbf{b}_i\big) \tag{12}$$
$$\mathbf{o}_t = \sigma\big(\mathbf{W}_o\,\mathbf{x}_t + \mathbf{U}_o\,\mathbf{h}_{t-1} + \mathbf{b}_o\big) \tag{13}$$
$$\tilde{\mathbf{c}}_t = \tanh\big(\mathbf{W}_c\,\mathbf{x}_t + \mathbf{U}_c\,\mathbf{h}_{t-1} + \mathbf{b}_c\big) \tag{14}$$
$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \tag{15}$$
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \tag{16}$$

The forward and backward hidden states are concatenated: $[\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] \in \mathbb{R}^{2d_h}$, then projected to vocabulary logits. Generation uses *prefix-based decoding*: the entire generated prefix is re-fed at each step.

Trainable parameters. Embedding (1,536) + LSTM Layer 1 bidir (198,656) + LSTM Layer 2 bidir (395,264) + Output (6,168) = **601,624**.

3.3 RNN with Attention (AttentionRNN)

This model follows an encoder–decoder design, both using **Elman RNN cells**. The encoder is a 2-layer RNN producing encoder outputs $\{\mathbf{e}_1, \ldots, \mathbf{e}_T\}$. At each decoding step t, Bahdanau-style additive attention with causal masking computes:

$$e_{t,i} = \mathbf{v}^\top \tanh\big(\mathbf{W}_K\,\mathbf{e}_i + \mathbf{W}_Q\,\mathbf{h}_t^{(\mathrm{dec})}\big), \quad i \le t \tag{17}$$
$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{t} \exp(e_{t,j})} \tag{18}$$
$$\mathbf{c}_t = \sum_{i=1}^{t} \alpha_{t,i}\,\mathbf{e}_i \tag{19}$$

where $\mathbf{W}_K, \mathbf{W}_Q \in \mathbb{R}^{d_h \times d_h}$ and scores for $i > t$ are set to $-\infty$. The context vector is concatenated with the input embedding and fed to the decoder RNN:

$$\mathbf{h}_t^{(\mathrm{dec})} = \tanh\big(\mathbf{W}_{xh}^{(\mathrm{dec})}\,[\mathbf{x}_t;\,\mathbf{c}_t] + \mathbf{W}_{hh}^{(\mathrm{dec})}\,\mathbf{h}_{t-1}^{(\mathrm{dec})} + \mathbf{b}_h^{(\mathrm{dec})}\big) \tag{20}$$

Trainable parameters. Embedding (1,536) + Encoder RNN L1 (24,832) + Encoder RNN L2 (33,024) + Attention (32,896) + Decoder RNN L1 (41,216) + Decoder RNN L2 (33,024) + Output (3,096) = **169,624**.

3.4 Hyperparameters

Table 9: Shared hyperparameters for all three RNN models.

Hyperparameter	**Value**
Embedding dimension (d_e)	64
Hidden size (d_h)	128
Number of layers	2
Dropout probability	0.2
Learning rate (Adam)	0.003
Training epochs	100
Teacher forcing ratio	0.5
Batch size	64
Gradient clipping (max norm)	5

4 Training Details

All models are trained with cross-entropy loss (ignoring `<PAD>`):

$$\mathcal{L} = -\frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n}\log P_\theta(y_t^{(n)} \mid y_{<t}^{(n)}) \tag{21}$$

using Adam ($\eta = 0.003$) with gradient clipping at norm 5. Teacher forcing is applied at the batch level with probability 0.5.

Table 10: Training loss at selected epochs and total training time.

Model	**Ep. 1**	**Ep. 20**	**Ep. 40**	**Ep. 60**	**Ep. 80**	**Ep. 100**	**Time (s)**
Vanilla RNN	2.2952	1.0502	1.0221	1.0270	1.0128	1.0140	29.6
BLSTM	2.2720	0.0010	0.0003	0.0002	0.0001	0.0001	152.0
Attention RNN	2.2720	0.6948	0.5187	0.5046	0.5496	0.4934	104.1

The BLSTM converges fastest to near-zero loss, indicating full memorisation of the training set. The Attention RNN converges to a moderate loss of 0.4934, and the Vanilla RNN plateaus around 1.01.

5 Quantitative Evaluation

We generate 300 names from each model and compute two metrics:

$$\text{Novelty} = \frac{|\{n \in \mathcal{G} : n \notin \mathcal{T}\}|}{|\mathcal{G}|}, \qquad \text{Diversity} = \frac{|\text{unique}(\mathcal{G})|}{|\mathcal{G}|} \tag{22}$$

Generation uses temperature scaling ($T = 1.0$), top-k sampling ($k = 8$), minimum length 4, hard-blocking of immediate repetition, and a soft repetition penalty of 0.6.

Table 11: Quantitative evaluation results (300 generated names per model).

Model	**Novelty (%)**	**Diversity (%)**	**Parameters**
Vanilla RNN	21.3	73.3	62,488
BLSTM	68.0	38.3	601,624
Attention RNN	21.0	47.0	169,624

The Vanilla RNN produces the most diverse output (73.3%) but mostly reconstructs training names (21.3% novel). The BLSTM achieves the highest novelty (68.0%) as stochastic decoding deviates from its memorised patterns, but diversity is low (38.3%). The Attention RNN balances structure with 21.0% novelty and 47.0% diversity.

6 Qualitative Analysis

6.1 Generated Samples

Table 12: Sample generated names from each model.

#	**Vanilla RNN**	**BLSTM**	**Attention RNN**
1	Dhruvir	Udip	Karesh
2	Ashendra	Udit	Dhruyash
3	Adiran	Urjud	Navdev
4	Devpal	Juyu	Sarvir
5	Surdedp	Urju	Karith
6	Varansh	Urjuy	Kariv
7	Sidpal	Juysh	Varsen
8	Maniv	Udiv	Karmeit
9	Rajpal	Juyu	Shivraj
10	Shivpal	Uvit	Dhruant
11	Sarmeh	Udit	Shivman
12	Surpal	Udiv	Karvir
13	Praith	Guraj	Varnath
14	Manvir	Jayit	Varith
15	Mahiv	Juyh	Dhruit

The **Vanilla RNN** produces phonetically natural names (*Dhruvir*, *Ashendra*, *Varansh*, *Rajpal*, *Manvir*) closely following training data patterns, with a preference for the *-pal* suffix. The **BLSTM** collapses onto a narrow set of patterns dominated by *U-* and *Ju-* prefixes, with many repeats (*Udit* appears three times in the top 15). The **Attention RNN** produces the most consistently realistic names (*Karesh*, *Dhruyash*, *Shivraj*, *Dhruant*, *Varnath*), with effective prefix–suffix combinations driven by the attention mechanism.

6.2 Failure Mode Analysis

Table 13: Failure mode analysis (out of 300 generated names).

Failure Type	Vanilla RNN	BLSTM	Attention RNN
Repeated characters (≥ 3)	0 (0.0%)	0 (0.0%)	0 (0.0%)
Incomplete (≤ 2 chars)	0 (0.0%)	0 (0.0%)	0 (0.0%)
Unnatural consonant clusters	2 (0.7%)	0 (0.0%)	3 (1.0%)

Generation heuristics eliminate character repetition and short names. Only the Vanilla RNN (e.g., *sadprjueman*) and Attention RNN (e.g., *krasendvr*, *kshraj*) produce occasional unnatural consonant clusters.

7 Character Embedding Visualisation

The learned 64-dimensional character embeddings are projected to 2D using PCA and t-SNE (perplexity=5, 2000 iterations). Vowels are plotted in red and consonants in blue.

In the PCA projections, the Vanilla RNN (PC1: 13.9%, PC2: 9.8%) shows partial vowel–consonant separation with overlap. The BLSTM (PC1: 11.1%, PC2: 10.6%) separates `i` and `u` from the main cluster. The Attention RNN (PC1: 11.5%, PC2: 10.6%) groups vowels more tightly together.

The t-SNE projections reveal that the Attention RNN produces the most interpretable structure: vowels cluster in specific regions, and suffix-consonants (`s`, `h`, `n`) are positioned nearby. The attention mechanism forces characters with similar distributional roles into similar embedding regions, as it must explicitly decide which encoder positions to attend to.

8 Comparison and Discussion

Table 14: Summary comparison of all three RNN models.

Model	Params	Final Loss	Time (s)	Novelty	Diversity	Failures
Vanilla RNN	62,488	1.0140	29.6	21.3%	73.3%	2/300
BLSTM	601,624	0.0001	152.0	68.0%	38.3%	0/300
Attention RNN	169,624	0.4934	104.1	21.0%	47.0%	3/300

The **Vanilla RNN** (62K params) is lightest and fastest, producing the most diverse names—but mostly reproducing training examples. The **BLSTM** (602K params) memorises the training set completely, achieving high novelty through stochastic deviation but with low diversity and phonetically unusual outputs. The **Attention RNN** (170K params) produces the most consistently realistic names; its moderate loss indicates effective learning without full memorisation.

9 Problem 2: Conclusion

We compared three character-level RNN architectures for Indian name generation. The Vanilla RNN produces the most diverse output (73.3%), the BLSTM achieves the highest novelty (68.0%) through deviation from memorised patterns, and the Attention RNN generates the most realistic-sounding names with the best structural coherence. Embedding visualisations confirm that the attention mechanism learns the most structured character representations.

These results highlight that model capacity must be matched to dataset size: on a small, structured corpus, a moderate-capacity model with attention outperforms both simpler and more complex alternatives in generative quality.